

SOFTWARE ENGINEERING LAB

EXPERIMENT 2

Selecting a Software Process Model

Name	Akanksha Upadhyay
Roll No.	20251651011
Date	27 August 2026
Project	TeamFinder

1. Aim

To compare software process models and select a suitable model for different software projects.

2. Process Model Comparison

Factor	Waterfall	Incremental	Prototype	Spiral	Agile
Requirements	Stable	Partly stable	Unclear	Can change	Changing
Early delivery	Low	Yes	Sample	Possible	Yes
Customer feedback	Low	Medium	High	High	Very high
Risk handling	Low	Medium	Medium	Very high	Medium

The comparison follows the factors specified for this laboratory experiment.

3. Case 1 – College Payroll System

Project characteristics: Salary rules, allowances, and deductions are already known, and major changes are not expected during development.

- Requirements: Stable and clearly defined.
- Risk: Relatively low because the rules and expected system behavior are known.
- Frequent customer feedback: Low need; requirements are already established.
- Possible choice: Waterfall Model.
- Reason: Waterfall is suitable when requirements are stable and changes are unlikely. The project can proceed through clearly defined phases such as requirements, design, implementation, testing, and deployment.

4. Case 2 – New Food-Delivery App

Project characteristics: The startup wants frequent releases, and users may request new features after trying the app.

- Requirements: Changing and expected to evolve.
- Early working software: Very important because the startup wants frequent releases.
- Customer feedback: Very high importance because user feedback will influence future features.
- Possible choice: Agile Model.
- Reason: Agile supports short development cycles, frequent releases, and continuous customer feedback. New features can be prioritized and implemented in later iterations.

5. Case 3 – Hospital Emergency Monitoring System

Project characteristics: Failure can have serious consequences, and technical and safety risks must be studied carefully.

- Risk: Very high because system failure may affect emergency monitoring.
- System: Safety-critical and technically sensitive.
- Risk analysis: Continuous and systematic risk identification and mitigation are required.
- Possible choice: Spiral Model.
- Reason: Spiral is appropriate when risk analysis is a major concern. Each cycle allows the team to identify, analyze, and reduce risks before moving further into development.

6. Case 4 – New College Portal Interface

Project characteristics: Students and staff are not sure how the new screens should look and want to see sample screens before finalising requirements.

- Requirements: Unclear and need clarification.
- Possible choice: Prototype Model.
- Reason: Prototyping allows sample screens to be created early. Students and staff can review the prototype, give feedback, and help the team refine the interface requirements before final development.

7. Case 5 – TeamFinder

Project characteristics: TeamFinder is a student collaboration platform for finding compatible teammates for projects and hackathons. Features such as skill-based matching, project posting, collaboration requests, profiles, and the user interface can be refined after student feedback.

- Requirements: Changing and likely to evolve as students use the platform and suggest improvements.
- Early working software: Important; the team can release core features first and add advanced features later.
- Customer/user feedback: Very high importance because the target users are students.
- Risk: Medium; usability, matching quality, and feature priorities can be improved through iterations.
- Possible choice: Agile Model.
- Reason 1: Agile supports changing requirements and allows TeamFinder to develop features in small iterations.
- Reason 2: Student feedback can be collected after each iteration and used to improve matching, profiles, project posting, and the interface.
- Reason 3: The project can deliver an MVP first—such as registration, profiles, and teammate search—and then incrementally add collaboration requests, hackathon teams, and other features.

8. Selected Models – Summary

Case	Selected Model	Main Reason
College Payroll System	Waterfall	Stable requirements and low expected change.
New Food-Delivery App	Agile	Frequent releases and continuous user feedback.
Hospital Emergency Monitoring System	Spiral	Very high risk and need for systematic risk analysis.
New College Portal Interface	Prototype	Unclear interface requirements require early samples and feedback.
TeamFinder	Agile	Changing requirements, iterative delivery, and strong student feedback.

9. Conclusion

Different software projects require different process models depending on requirements, risk, delivery expectations, and customer feedback. Waterfall is suitable for the stable payroll case, Agile for the evolving food-delivery application, Spiral for the high-risk hospital monitoring system, and Prototype for the uncertain college portal interface. For TeamFinder, Agile is the most suitable model because its requirements can evolve with student feedback and the

system can be developed and released incrementally.

10. Result

The five project cases were analyzed, the five software process models were compared, and the most suitable process model was selected for each case with justification. Agile was selected for TeamFinder.